Temas Avanzados de Regresión

Actividad Práctica: No-linealidad, Heteroscedasticidad y Outliers

In [1…

```
#!pip install statsmodels
```

In [4…

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn import metrics

import statsmodels.api as sm
import statsmodels.stats.api as sms
from statsmodels.stats.diagnostic import het_breuschpagan, het_white
from statsmodels.stats.outliers_influence import variance_inflation_factor, OLSInfluence

%matplotlib inline
```

In [8…

```
# Datos Real Estate
df = pd.read_csv('data/RealEstate.csv')

# Renombrar columnas
df.columns = ['No', 'Fecha_Transaccion', 'Antiguedad', 'Distancia_MRT',
              'Tiendas_Conveniencia', 'Latitud', 'Longitud', 'Precio']

print("Datos cargados exitosamente")
print(f"Forma del dataset: {df.shape}")
df.head()
```

```
Datos cargados exitosamente
Forma del dataset: (414, 8)
```

Out[8…

	No	Fecha_Transaccion	Antiguedad	Distancia_MRT	Tiendas_Conveniencia	Latitud	Longitud	Precio
0	1	2012.917	32.0	84.87882	10	24.98298	121.54024	37.9
1	2	2012.917	19.5	306.59470	9	24.98034	121.53951	42.2
2	3	2013.583	13.3	561.98450	5	24.98746	121.54391	47.3
3	4	2013.500	13.3	561.98450	5	24.98746	121.54391	54.8
4	5	2012.833	5.0	390.56840	5	24.97937	121.54245	43.1

In [9…

```
# Análisis exploratorio inicial
print("=== ESTADÍSTICAS DESCRIPTIVAS ===\n")
print(df.describe().round(2))
```

```
=== ESTADÍSTICAS DESCRIPTIVAS ===

           No  Fecha_Transaccion  Antiguedad  Distancia_MRT  \
count  414.00             414.00      414.00         414.00
mean   207.50            2013.15       17.71        1083.89
std    119.66               0.28       11.39        1262.11
min      1.00            2012.67        0.00          23.38
25%    104.25            2012.92        9.02         289.32
50%    207.50            2013.17       16.10         492.23
75%    310.75            2013.42       28.15        1454.28
max    414.00            2013.58       43.80        6488.02

       Tiendas_Conveniencia  Latitud  Longitud  Precio
count                414.00   414.00    414.00  414.00
mean                   4.09    24.97    121.53   37.98
std                    2.95     0.01      0.02   13.61
min                    0.00    24.93    121.47    7.60
25%                    1.00    24.96    121.53   27.70
50%                    4.00    24.97    121.54   38.45
75%                    6.00    24.98    121.54   46.60
max                   10.00    25.01    121.57  117.50
```

In [1…

```
# Visualización de relaciones no-lineales
# Nota como cambia log(Distancia MRT) y sin la transformación
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

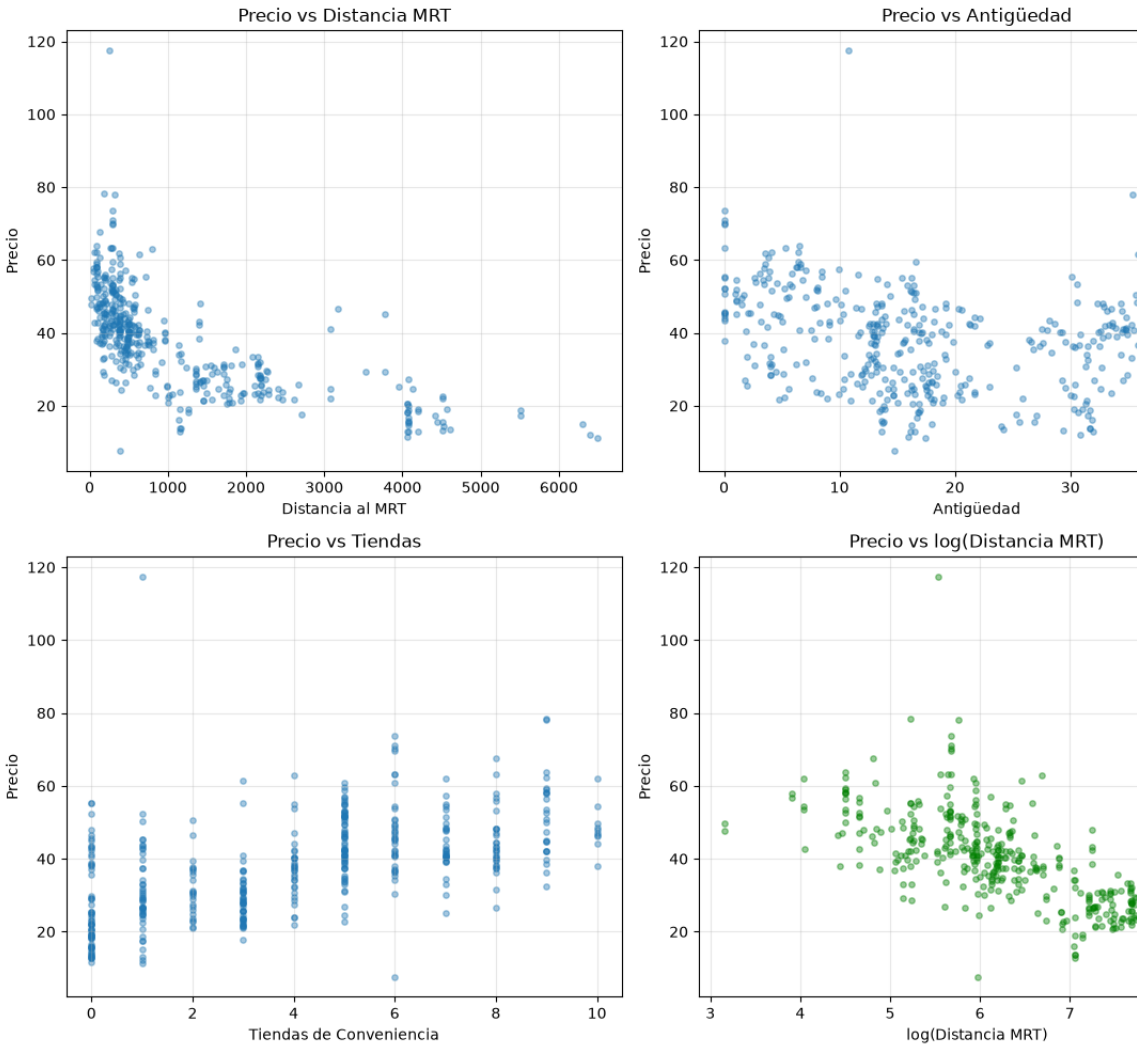
# Precio vs Distancia MRT
axes[0, 0].scatter(df['Distancia_MRT'], df['Precio'], alpha=0.4, s=15)
axes[0, 0].set_xlabel('Distancia al MRT')
axes[0, 0].set_ylabel('Precio')
axes[0, 0].set_title('Precio vs Distancia MRT')
axes[0, 0].grid(True, alpha=0.3)

# Precio vs Antigüedad
axes[0, 1].scatter(df['Antiguedad'], df['Precio'], alpha=0.4, s=15)
axes[0, 1].set_xlabel('Antigüedad')
axes[0, 1].set_ylabel('Precio')
axes[0, 1].set_title('Precio vs Antigüedad')
axes[0, 1].grid(True, alpha=0.3)

# Precio vs Tiendas
axes[1, 0].scatter(df['Tiendas_Conveniencia'], df['Precio'], alpha=0.4, s=15)
axes[1, 0].set_xlabel('Tiendas de Conveniencia')
axes[1, 0].set_ylabel('Precio')
axes[1, 0].set_title('Precio vs Tiendas')
axes[1, 0].grid(True, alpha=0.3)

# Precio vs log(Distancia MRT)
axes[1, 1].scatter(np.log(df['Distancia_MRT']), df['Precio'], alpha=0.4, s=15, color='green')
#axes[1, 1].scatter(df['Distancia_MRT'], df['Precio'], alpha=0.4, s=15, color='green')
#axes[1, 1].set_xlabel('Distancia MRT')
axes[1, 1].set_xlabel('log(Distancia MRT)')
axes[1, 1].set_ylabel('Precio')
axes[1, 1].set_title('Precio vs log(Distancia MRT)')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



In […

In […

PREGUNTAS DE ANÁLISIS - NO-LINEALIDAD:

1. ¿Qué variables muestran una relación no-lineal con el Precio?"

La distancia MRT al igual que la Antigüedad presentan una aparente distribución no lineal y tiendas de conveniencia no parecer seguir algún tipo de distribución y los precios solo pare estar dispersos entre las tiendas de conveniencia.

1. ¿La transformación logarítmica mejora la linealidad de la Distancia MRT? (Puedes com las lineas con log y sin el log para ver la diferencia)

En la distancia antes de transformarla logaritmicamente se puede notar una caída levement pronunciada que comienza desde los primeros valores en el eje X y se va suavizando conf avanza sobre el mismo eje, por lo que se puede observar claramente una línea de tendenci logarítmica, en cambio al pasar a una transformación logarítmica se puede ver que la caída línea de tendencia no se realentiza en ningún momento y puede verse una caída contínua.

1. ¿Qué tipo de transformación propondrías para cada variable?

Para la antigüedad propondría una transformación polinómica ya que parece ser que en un de la gráfica los puntos caen o bajan el precio y vuelven a subir, como si trataran de seguir parábola levemente pronunciada o algún polinomio que funcione, en cambio en las tiendas conveniencia no parece existir algún tipo de regresión que pueda describir el comportamier scatter plot.

In [1…

```python
# Comparar modelos con y sin transformación
X = df.drop(['Precio', 'No'], axis=1)
y = df['Precio']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Modelo lineal con Distancia MRT
modelo_lineal = LinearRegression()
modelo_lineal.fit(X_train[['Distancia_MRT']], y_train)
y_pred_lineal = modelo_lineal.predict(X_test[['Distancia_MRT']])

# Modelo logarítmico con Distancia MRT
X_train_log = np.log(X_train[['Distancia_MRT']])
X_test_log = np.log(X_test[['Distancia_MRT']])

modelo_log = LinearRegression()
modelo_log.fit(X_train_log, y_train)
y_pred_log = modelo_log.predict(X_test_log)

# Comparar R²
print("=== COMPARACIÓN DE MODELOS ===")
print(f"Modelo Lineal (Distancia MRT):")
print(f"  R² = {metrics.r2_score(y_test, y_pred_lineal):.4f}")
print(f"  RMSE = {np.sqrt(metrics.mean_squared_error(y_test, y_pred_lineal)):.4f}")
print(f"\nModelo Logarítmico (log Distancia MRT):")
print(f"  R² = {metrics.r2_score(y_test, y_pred_log):.4f}")
print(f"  RMSE = {np.sqrt(metrics.mean_squared_error(y_test, y_pred_log)):.4f}")
```

```
=== COMPARACIÓN DE MODELOS ===
Modelo Lineal (Distancia MRT):
  R² = 0.4277
  RMSE = 9.7827

Modelo Logarítmico (log Distancia MRT):
  R² = 0.5053
  RMSE = 9.0952
```

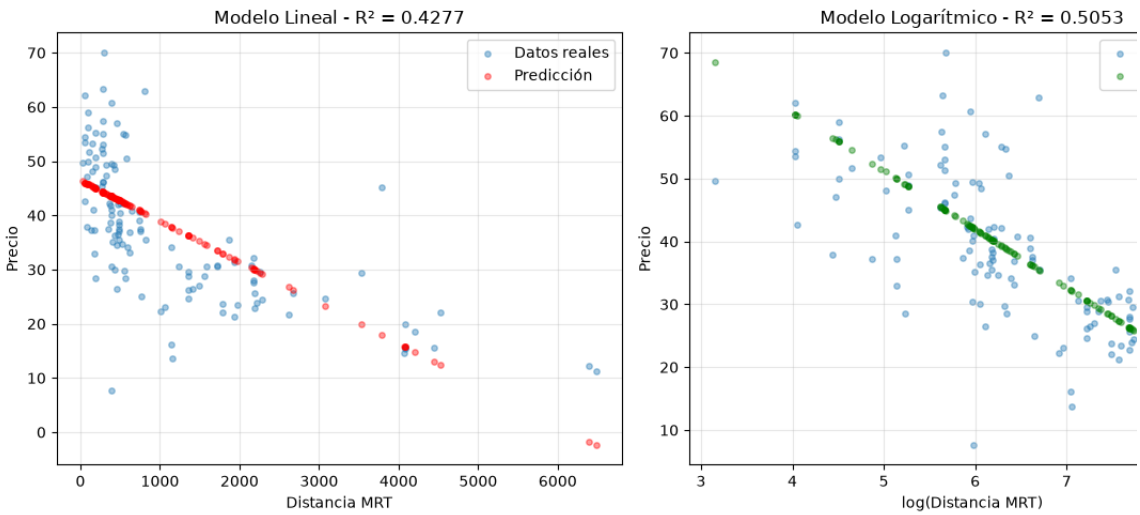
In [1…

```python
# Visualizar la mejora de la transformación
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Modelo lineal
axes[0].scatter(X_test['Distancia_MRT'], y_test, alpha=0.4, s=15, label='Datos reales')
axes[0].scatter(X_test['Distancia_MRT'], y_pred_lineal, alpha=0.4, s=15, color='red',
label='Predicción')
axes[0].set_xlabel('Distancia MRT')
axes[0].set_ylabel('Precio')
axes[0].set_title(f'Modelo Lineal - R² = {metrics.r2_score(y_test, y_pred_lineal):.4f}')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Modelo logarítmico
axes[1].scatter(X_test_log, y_test, alpha=0.4, s=15, label='Datos reales')
axes[1].scatter(X_test_log, y_pred_log, alpha=0.4, s=15, color='green', label='Predicción')
axes[1].set_xlabel('log(Distancia MRT)')
axes[1].set_ylabel('Precio')
axes[1].set_title(f'Modelo Logarítmico - R² = {metrics.r2_score(y_test, y_pred_log):.4f}')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



In [1…

```python
#  Regresión polinómica de diferentes grados
print("=== REGRESIÓN POLINÓMICA ===")

grados = [1, 2, 3, 4]
resultados = []

for grado in grados:
    # Crear pipeline con transformación polinómica
    modelo_poly = make_pipeline(
        PolynomialFeatures(degree=grado, include_bias=False),
        LinearRegression()
    )

    # Entrenar
    modelo_poly.fit(X_train[['Distancia_MRT']], y_train)
    y_pred_poly = modelo_poly.predict(X_test[['Distancia_MRT']])

    # Métricas
    r2 = metrics.r2_score(y_test, y_pred_poly)
    rmse = np.sqrt(metrics.mean_squared_error(y_test, y_pred_poly))
    aic = len(y_test) * np.log(metrics.mean_squared_error(y_test, y_pred_poly)) + 2 * (grado + 1)

    resultados.append({
        'Grado': grado,
        'R²': r2,
        'RMSE': rmse,
        'AIC': aic
    })

df_resultados = pd.DataFrame(resultados)
print(df_resultados.round(4))
```

```
=== REGRESIÓN POLINÓMICA ===
   Grado      R²     RMSE       AIC
0      1  0.4277   9.7827  574.1541
1      2  0.4789   9.3353  564.4499
2      3  0.3128  10.7201  601.0312
3      4  0.2282  11.3604  617.5336
```

In [1…

```python
# Visualizar regresión polinómica
plt.figure(figsize=(12, 5))

# Gráfico de datos
plt.subplot(1, 2, 1)
plt.scatter(X_test['Distancia_MRT'], y_test, alpha=0.4, s=15, label='Datos Real E.')

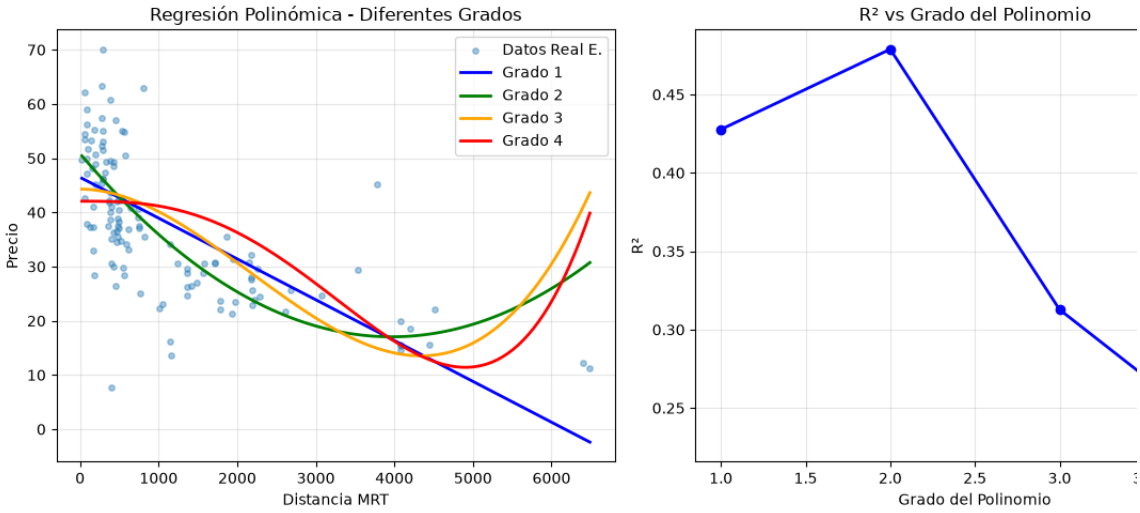
# Crear rango para predicciones suaves
X_range = np.linspace(X_test['Distancia_MRT'].min(), X_test['Distancia_MRT'].max(), 100).reshape(-1, 1)

for grado, color in zip([1, 2, 3, 4], ['blue', 'green', 'orange', 'red']):
    modelo_poly = make_pipeline(
        PolynomialFeatures(degree=grado, include_bias=False),
        LinearRegression()
    )
    modelo_poly.fit(X_train[['Distancia_MRT']], y_train)
    y_range = modelo_poly.predict(X_range)
    plt.plot(X_range, y_range, color=color, linewidth=2, label=f'Grado {grado}')

plt.xlabel('Distancia MRT')
plt.ylabel('Precio')
plt.title('Regresión Polinómica - Diferentes Grados')
plt.legend()
plt.grid(True, alpha=0.3)

# Gráfico de R² vs Grado
plt.subplot(1, 2, 2)
plt.plot(df_resultados['Grado'], df_resultados['R²'], 'o-', color='blue', linewidth=2)
plt.xlabel('Grado del Polinomio')
plt.ylabel('R²')
plt.title('R² vs Grado del Polinomio')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



Preguntas de análisis - Polinomios

PREGUNTAS DE ANÁLISIS - POLINOMIOS:

1. ¿Qué grado de polinomio da el mejor R²?

El segundo grado del polinomio presenta una mejor R² a comparación de las otras regresio multinomiales ya que alcanza a explicar más del $ 4 5 \%$ de variabilidad de los datos, lo significa que sea un buen modelo pero si simboliza que es el mejor de los $ 4 $ presentadc

1. ¿Hay evidencia de sobreajuste con grados altos?

Si se puede notar la evidencia que el aumento de los grados están empeorando el modelo no podemos afirmar que exista un sobreajuste únicamente observando la gráfica anterior y tres criterios (R² RMSE AIC)

1. ¿Qué grado seleccionarías según AIC?"

Según este criterio debería seleccionarse el modelo que tenga el valor más pequeño del cri (en este caso segundo grado) ya que indica un mejor compromiso entre ajuste y complejida modelo.

In […

```
#  Modelo completo y análisis de residuos
X_const = sm.add_constant(X)
modelo_completo = sm.OLS(y, X_const).fit()
print(modelo_completo.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Precio   R-squared:                       0.582
Model:                             OLS   Adj. R-squared:                  0.576
Method:                  Least Squares   F-statistic:                     94.59
Date:                 Thu, 24 Sep 2026   Prob (F-statistic):           4.86e-74
Time:                         17:57:15   Log-Likelihood:                -1487.0
No. Observations:                  414   AIC:                             2988.
Df Residuals:                      407   BIC:                             3016.
Df Model:                            6
Covariance Type:            nonrobust
===========================================================================================
                              coef    std err          t      P>|t|      [0.025      0.975]
-------------------------------------------------------------------------------------------
const                -1.444e+04   6775.671     -2.131      0.034   -2.78e+04   -1117.421
Fecha_Transaccion        5.1462      1.557      3.305      0.001       2.085       8.207
Antiguedad              -0.2697      0.039     -7.000      0.000      -0.345      -0.194
Distancia_MRT           -0.0045      0.001     -6.250      0.000      -0.006      -0.003
Tiendas_Conveniencia     1.1333      0.188      6.023      0.000       0.763       1.503
Latitud                225.4730     44.567      5.059      0.000     137.863     313.083
Longitud               -12.4236     48.582     -0.256      0.798    -107.927      83.079
==============================================================================
Omnibus:                      231.634   Durbin-Watson:                   2.153
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             3599.138
Skew:                           2.026   Prob(JB):                         0.00
Kurtosis:                      16.865   Cond. No.                     3.72e+07
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 3.72e+07. This might indicate that there are
strong multicollinearity or other numerical problems.
```

In [1…

```python
# Visualización de residuos vs predichos
residuos = modelo_completo.resid
predichos = modelo_completo.fittedvalues

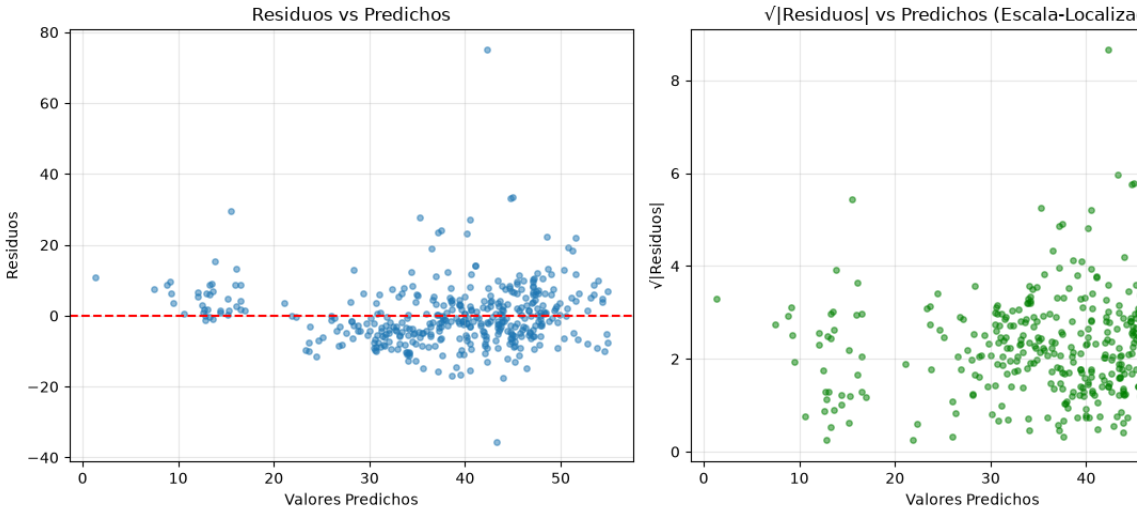
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(predichos, residuos, alpha=0.5, s=15)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('Valores Predichos')
plt.ylabel('Residuos')
plt.title('Residuos vs Predichos')
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.scatter(predichos, np.sqrt(np.abs(residuos)), alpha=0.5, s=15, color='green')
plt.xlabel('Valores Predichos')
plt.ylabel('√|Residuos|')
plt.title('√|Residuos| vs Predichos (Escala-Localización)')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("PREGUNTA: ¿Observas un patrón en forma de embudo?")
```




```
PREGUNTA: ¿Observas un patrón en forma de embudo?
```

Escribe tu respuesta aquí:

A simple vista no se puede observar un patrón de embudo ya que los residuos graficados s muestran muy dispersos

In [1…

```python
# Prueba de Breusch-Pagan
print("=== PRUEBA DE BREUSCH-PAGAN ===\n")

bp_test = het_breuschpagan(modelo_completo.resid, X_const)
print(f"LM Statistic: {bp_test[0]:.4f}")
print(f"LM p-value: {bp_test[1]:.4f}")
print(f"F Statistic: {bp_test[2]:.4f}")
print(f"F p-value: {bp_test[3]:.4f}")

if bp_test[1] < 0.05:
    print("\n❌ Rechazamos H₀: Hay evidencia de heteroscedasticidad")
else:
    print("\n✅ No rechazamos H₀: No hay evidencia suficiente de heteroscedasticidad")
```

```
=== PRUEBA DE BREUSCH-PAGAN ===

LM Statistic: 8.4591
LM p-value: 0.2064
F Statistic: 1.4149
F p-value: 0.2074

✅ No rechazamos H₀: No hay evidencia suficiente de heteroscedasticidad
```

PREGUNTAS DE ANÁLISIS - HETEROSCEDASTICIDAD:

1. ¿Qué concluyes de las pruebas de Breusch-Pagan y White?

Solo se puede concluir en que no existe evidencia suficiente para la heteroscedasticidad ya no se presenta evidencia estadísticamente significativa de cambiar sistemáticamente con la variables explicadas del modelo

1. ¿Los resultados son consistentes con el análisis visual?

Si son consistentes porque visualmente no hay señal fuerte de heteroscedasticidad

1. ¿Qué implicaciones tiene para la inferencia?

Si se asumen que los demás supuestos relevantes del modelo son razinables la ausencia d evidencia respalda el uso de errores estándar.

In [1…

```
# Errores estándar robustos (Sandwich)
print("=== SOLUCIÓN 1: ERRORES ESTÁNDAR ROBUSTOS ===\n")

# Modelo con errores robustos HC3
modelo_robusto = sm.OLS(y, X_const).fit(cov_type='HC3')

print("Comparación de errores estándar:")
print(f"{'Variable':<20} {'MCO':<15} {'Robusto':<15} {'Cambio %':<10}")
print("-" * 60)

for var in X_const.columns:
    se_mco = modelo_completo.bse[var]
    se_rob = modelo_robusto.bse[var]
    cambio = (se_rob - se_mco) / se_mco * 100
    print(f"{var:<20} {se_mco:<15.4f} {se_rob:<15.4f} {cambio:>+8.2f}%")

print("\n PREGUNTA: ¿Cómo cambian los errores estándar?")

# Escribe tu respuesta aquí
```

```
=== SOLUCIÓN 1: ERRORES ESTÁNDAR ROBUSTOS ===

Comparación de errores estándar:
Variable             MCO             Robusto         Cambio %
------------------------------------------------------------
const                6775.6707       5598.1457         -17.38%
Fecha_Transaccion    1.5571          1.4696             -5.62%
Antiguedad           0.0385          0.0431            +11.73%
Distancia_MRT        0.0007          0.0008            +16.36%
Tiendas_Conveniencia 0.1882          0.2317            +23.13%
Latitud              44.5667         46.7949            +5.00%
Longitud             48.5820         44.1002            -9.23%

 PREGUNTA: ¿Cómo cambian los errores estándar?
```

Escribe tu respuesta aquí: Los errores cambian de manera diferente para cada variable a u errores estándar robustos porque estos mismos corrigen la estimación de la incertidumbre.

In [1…

```python
#  Mínimos Cuadrados Ponderados (WLS)
print("=== SOLUCIÓN 2: MÍNIMOS CUADRADOS PONDERADOS (WLS) ===\n")

# Estimar la varianza usando los residuos al cuadrado
# Modelo auxiliar: log(e²) ~ X
residuos_sq = modelo_completo.resid ** 2
log_residuos_sq = np.log(residuos_sq + 1e-10)  # Evitar log(0)

modelo_aux = sm.OLS(log_residuos_sq, X_const).fit()
var_predicha = np.exp(modelo_aux.fittedvalues)

# Pesos = 1/varianza
pesos = 1 / var_predicha

# Modelo WLS
modelo_wls = sm.WLS(y, X_const, weights=pesos).fit()

print("Comparación de coeficientes:")
print(f"{'Variable':<20} {'MCO':<15} {'WLS':<15} {'Cambio %':<10}")
print("-" * 60)

for var in X_const.columns:
    coef_mco = modelo_completo.params[var]
    coef_wls = modelo_wls.params[var]
    cambio = (coef_wls - coef_mco) / abs(coef_mco) * 100
    print(f"{var:<20} {coef_mco:<15.4f} {coef_wls:<15.4f} {cambio:>+8.2f}%")

print("\n PREGUNTA: ¿Cómo cambian los coeficientes estimados?")

# Escribe tu respuesta aquí
```

```
=== SOLUCIÓN 2: MÍNIMOS CUADRADOS PONDERADOS (WLS) ===

Comparación de coeficientes:
Variable             MCO             WLS             Cambio %
------------------------------------------------------------
const                -14437.1008     -10816.4160       +25.08%
Fecha_Transaccion    5.1462          4.3956            -14.59%
Antiguedad           -0.2697         -0.3029           -12.32%
Distancia_MRT        -0.0045         -0.0038           +15.81%
Tiendas_Conveniencia 1.1333          1.3931            +22.93%
Latitud              225.4730        227.2422           +0.78%
Longitud             -12.4236        -30.1560         -142.73%

 PREGUNTA: ¿Cómo cambian los coeficientes estimados?
```

Escribe tu respuesta aquí: En este caso cambiar a MCO si cambian los coeficientes estima cambio al anterior que los coeficientes permanecían iguales.

In [2…

```
# Comparación de modelos con diferentes soluciones
print("=== COMPARACIÓN DE SOLUCIONES ===\n")

print("Modelo MCO (sin corrección):")
print(f"  R² = {modelo_completo.rsquared:.4f}")
print(f"  AIC = {modelo_completo.aic:.2f}")
print(f"  BIC = {modelo_completo.bic:.2f}")

print("\nModelo Robusto (HC3):")
print(f"  R² = {modelo_robusto.rsquared:.4f}")
print(f"  AIC = {modelo_robusto.aic:.2f}")
print(f"  BIC = {modelo_robusto.bic:.2f}")

print("\nModelo WLS:")
print(f"  R² = {modelo_wls.rsquared:.4f}")
print(f"  AIC = {modelo_wls.aic:.2f}")
print(f"  BIC = {modelo_wls.bic:.2f}")
```

```
=== COMPARACIÓN DE SOLUCIONES ===

Modelo MCO (sin corrección):
  R² = 0.5824
  AIC = 2987.92
  BIC = 3016.11

Modelo Robusto (HC3):
  R² = 0.5824
  AIC = 2987.92
  BIC = 3016.11

Modelo WLS:
  R² = 0.7060
  AIC = 2919.51
  BIC = 2947.69
```

PREGUNTAS DE ANÁLISIS - SOLUCIONES:

1. ¿Cuál solución prefieres y por qué?

el WLS presenta un ajuste aparentemente superior a comparación de los otros modelos por sus criterior AIC y BIC son menores y explica el $ 7 0 /%$ de la variabilidad

1. ¿Los errores robustos cambian los coeficientes? ¿Por qué?

Los errores estándar robustos no vuelven a estimar el modelo, lo único que cambian es la estimación de la matriz de varianzas-covarianzas

1. ¿Qué ventaja tiene WLS sobre los errores robustos?

La ventaja principal es que WLS intenta modelar explícitamente la heterocedasticidad mient que HC 3 simplemente hace que la inferencia sea más resistente a ella.

In [2…

```python
#  Detección de outliers
print("=== DETECCIÓN DE OUTLIERS ===\n")

# Calcular influencia
influencia = OLSInfluence(modelo_completo)

# Distancia de Cook
cook_dist = influencia.cooks_distance[0]
umbral_cook = 4 / len(y)

# Leverage
leverage = influencia.hat_matrix_diag
umbral_leverage = 2 * X_const.shape[1] / len(y)

# Residuos estandarizados
residuos_est = influencia.resid_studentized_internal

# Identificar outliers
outliers_cook = np.where(cook_dist > umbral_cook)[0]
outliers_leverage = np.where(leverage > umbral_leverage)[0]
outliers_residuos = np.where(np.abs(residuos_est) > 3)[0]

print(f"Distancia de Cook > {umbral_cook:.4f}: {len(outliers_cook)} observaciones")
print(f"Leverage > {umbral_leverage:.4f}: {len(outliers_leverage)} observaciones")
print(f"|Residuos estandarizados| > 3: {len(outliers_residuos)} observaciones")
```

```
=== DETECCIÓN DE OUTLIERS ===

Distancia de Cook > 0.0097: 19 observaciones
Leverage > 0.0338: 15 observaciones
|Residuos estandarizados| > 3: 7 observaciones
```

In [2…

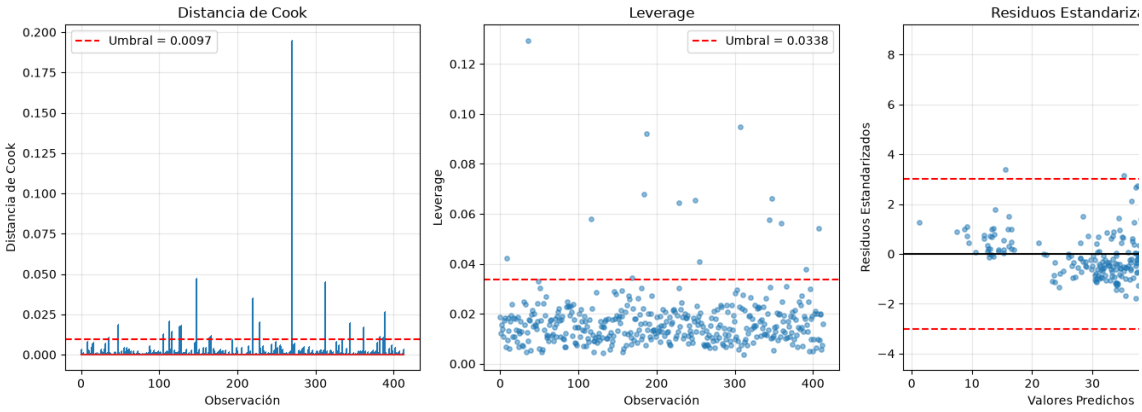
```python
# Visualización de outliers
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Distancia de Cook
axes[0].stem(range(len(cook_dist)), cook_dist, markerfmt=',')
axes[0].axhline(y=umbral_cook, color='r', linestyle='--', label=f'Umbral = {umbral_cook:.4f}')
axes[0].set_xlabel('Observación')
axes[0].set_ylabel('Distancia de Cook')
axes[0].set_title('Distancia de Cook')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Leverage
axes[1].scatter(range(len(leverage)), leverage, alpha=0.5, s=15)
axes[1].axhline(y=umbral_leverage, color='r', linestyle='--', label=f'Umbral = {umbral_leverage:.4f}')
axes[1].set_xlabel('Observación')
axes[1].set_ylabel('Leverage')
axes[1].set_title('Leverage')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# Residuos estandarizados
axes[2].scatter(modelo_completo.fittedvalues, residuos_est, alpha=0.5, s=15)
axes[2].axhline(y=3, color='r', linestyle='--')
axes[2].axhline(y=-3, color='r', linestyle='--')
axes[2].axhline(y=0, color='black', linestyle='-')
axes[2].set_xlabel('Valores Predichos')
axes[2].set_ylabel('Residuos Estandarizados')
axes[2].set_title('Residuos Estandarizados')
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



In [2…

```python
# Regresión robusta con Huber
print("=== REGRESIÓN ROBUSTA (HUBER) ===\n")

from sklearn.linear_model import HuberRegressor

# Modelo Huber
huber = HuberRegressor(epsilon=1.35, max_iter=1000)
huber.fit(X_train, y_train)

# Predicciones
y_pred_huber = huber.predict(X_test)

# Comparar con MCO
modelo_mco = LinearRegression()
modelo_mco.fit(X_train, y_train)
y_pred_mco = modelo_mco.predict(X_test)

print("Comparación de coeficientes:")
print(f"{'Variable':<25} {'MCO':<15} {'Huber':<15}")
print("-" * 55)

for i, var in enumerate(X.columns):
    print(f"{var:<25} {modelo_mco.coef_[i]:<15.4f} {huber.coef_[i]:<15.4f}")

print(f"\n{'Intercept':<25} {modelo_mco.intercept_:<15.4f} {huber.intercept_:<15.4f}")

print("\nMétricas en test:")
print(f"MCO:   R² = {metrics.r2_score(y_test, y_pred_mco):.4f}, RMSE =
{np.sqrt(metrics.mean_squared_error(y_test, y_pred_mco)):.4f}")
print(f"Huber: R² = {metrics.r2_score(y_test, y_pred_huber):.4f}, RMSE =
{np.sqrt(metrics.mean_squared_error(y_test, y_pred_huber)):.4f}")
```

```
=== REGRESIÓN ROBUSTA (HUBER) ===

Comparación de coeficientes:
Variable                  MCO             Huber
-------------------------------------------------------
Fecha_Transaccion         5.8478          0.0234
Antiguedad                -0.2425         -0.2380
Distancia_MRT             -0.0051         -0.0054
Tiendas_Conveniencia      1.0745          1.3118
Latitud                   239.0969        0.0423
Longitud                  -52.2352        -0.0528

Intercept                 -11350.7405     -0.0003

Métricas en test:
MCO:   R² = 0.5601, RMSE = 8.5772
Huber: R² = 0.5519, RMSE = 8.6567
```

In [2…

```python
#  RANSAC
print("=== RANSAC ===\n")

from sklearn.linear_model import RANSACRegressor

# Modelo RANSAC
ransac = RANSACRegressor(
    estimator=LinearRegression(),
    min_samples=0.5,
    residual_threshold=5.0,
    random_state=42
)
ransac.fit(X_train, y_train)

# Predicciones
y_pred_ransac = ransac.predict(X_test)

print(f"Número de inliers: {ransac.inlier_mask_.sum()}")
print(f"Número de outliers: {(~ransac.inlier_mask_).sum()}")

print("\nMétricas en test:")
print(f"RANSAC: R² = {metrics.r2_score(y_test, y_pred_ransac):.4f}, RMSE =
{np.sqrt(metrics.mean_squared_error(y_test, y_pred_ransac)):.4f}")
```

```
=== RANSAC ===

Número de inliers: 160
Número de outliers: 129

Métricas en test:
RANSAC: R² = 0.5225, RMSE = 8.9363
```

PREGUNTAS DE ANÁLISIS - OUTLIERS:

1. ¿Cuántos outliers detectaste con cada método?

Distancia de Cook: 19 observaciones influyentes. Leverage: 15 observacione valores de apalancamiento elevados. Residuos estandarizados: 7 observaciones RAN identificó 129 outliers y 160 inliers.

1. ¿Los coeficientes cambian significativamente con métodos robustos?

Sí, algunos coeficientes cambian considerablemente aunque no todos

1. ¿Qué método recomendarías para este dataset?

MCO presenta el mejor desempeño predictivo aunque no explica el total pero si ligeramente mayor parte de la variabilidad frente a HUBER